

Neural Networks II: Images and Convolutional Networks

Lecture 10 – Introduction to Modern Machine Learning

Steven Campbell

Imperial College London Summer School

June 2026

Outline

1. From MLPs to images
2. Convolutional layers
3. A small CNN for digits
4. Practical CNN training

From MLPs to images

Where this lecture fits

Lecture 9 introduced neural networks as differentiable functions trained by loss minimisation:

$$x \mapsto f_{\theta}(x), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

Earlier lectures also gave us:

- loss functions, generalisation and validation;
- regularisation and overfitting;
- representation learning through PCA;
- online updates and stochastic gradient methods.

Today

We ask how to adapt neural networks to **images**, where pixels have spatial structure.

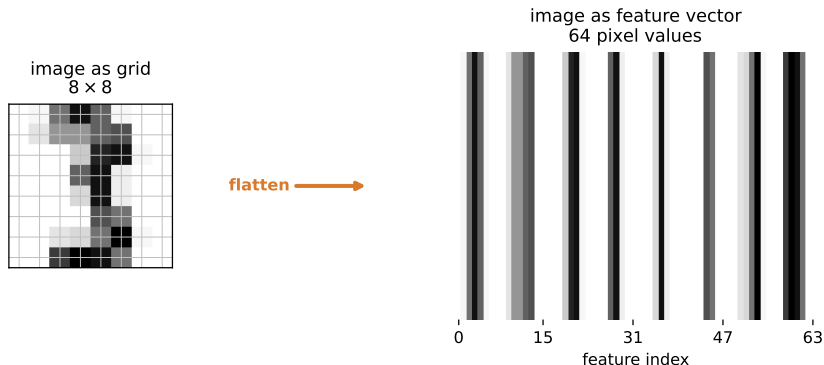
Recall: the basic neural-network training loop



Same training loop, different architecture

A Convolutional Neural Network (CNN) is still trained by forward pass $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ optimiser step. The new idea is the **convolutional layer**.

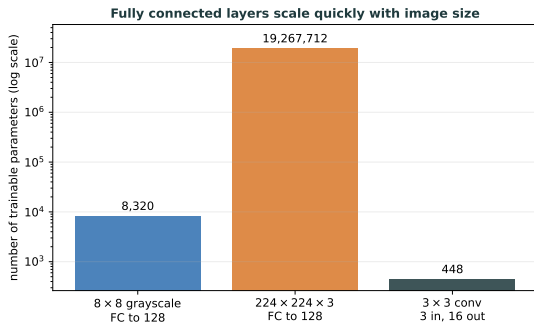
Images have spatial structure



Flattening an image into a vector is mathematically valid, but it hides the fact that neighbouring pixels are related.

Why not just use a fully connected network?

For a fully connected layer, every input pixel connects to every hidden unit.



Problem

It ignores locality and learns separate weights for each pixel-location relationship.

What we would like for images

Good image models should use three simple ideas:

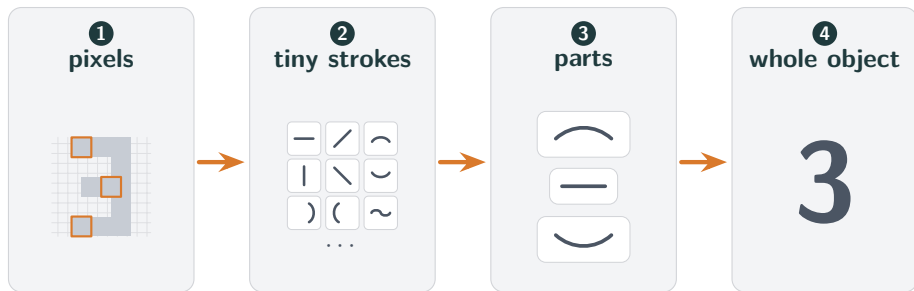
- **Locality:** nearby pixels form meaningful patterns such as edges, corners and strokes.
- **Weight sharing:** the same pattern can appear in many locations.
- **Hierarchy:** simple patterns combine into more complex patterns.



Convolutional neural networks build these ideas into the architecture.

From pixels to the whole object...

The big idea visualised:



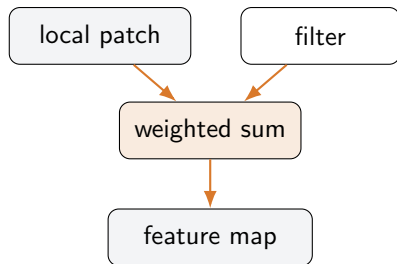
Convolutional layers

Convolution in one sentence

Convolutional layer

A convolutional layer learns small filters and applies the same filters across spatial locations.

- 1 Place a small filter on a local image patch.
- 2 Multiply matching entries and add them up.
- 3 Move the same filter to the next location.



Key idea

The layer does not learn a separate connection for every pixel-location relationship; it reuses a small set of weights.

A sliding-filter view of convolution

Convolution mathematically: shift, multiply, add

Classical definition

$$(f * g)(t) = \int f(u) g(t - u) du.$$

Read this as:

- flip and shift g ;
- multiply with f ;
- add up the overlap.

CNN code

For a 2D image X and filter K :

$$Y_{ij} = \sum_{u=1}^h \sum_{v=1}^w K_{uv} X_{i+u-1, j+v-1}.$$

This is the no-flip version, usually called *cross-correlation*.

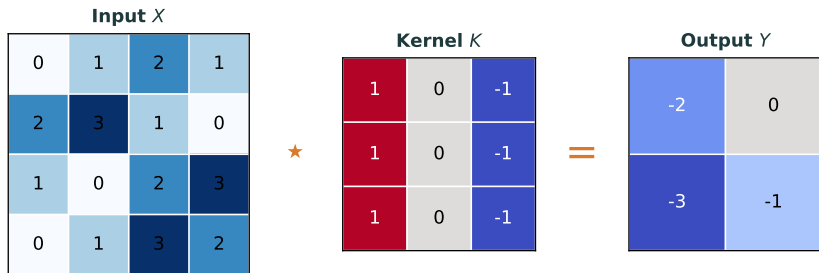
Operational view

For this lecture, think: **local patch** $\times$ **shared filter** $\rightarrow$ **one output value**.

Since the filter is learned, the flip convention rarely matters in practice.

A small 2D example

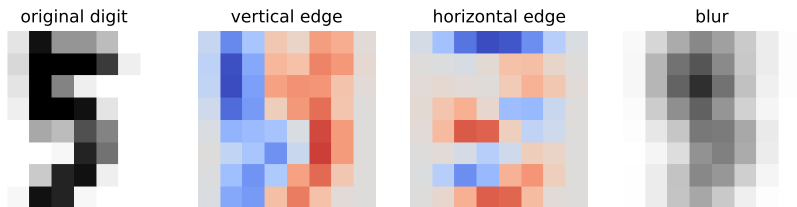
A 2D convolution/correlation computes local weighted sums



Each output entry is computed from one local window of the input.

Filters as pattern detectors

Filters act like pattern detectors



In a CNN

The filters are not usually chosen by hand. They are learned from data by backpropagation.

Feature maps

If a filter is applied at all locations, the output is called a **feature map**.



With multiple filters, we get multiple feature maps:

image $\longrightarrow$ stack of feature maps.

For an RGB image, filters also combine information across input channels (i.e., in this case, the colors).

Locality and weight sharing

Locality

Each output depends only on a small region of the input.

This reduces parameters and encodes the belief that local pixel patterns are meaningful.

Weight sharing

The same filter weights are reused at every spatial location.

This lets the model detect the same pattern wherever it appears in the image.

few shared weights



many locations

Equivariance and invariance

Translation equivariance (informal)

If an input pattern shifts, the corresponding feature-map response shifts.

Convolution preserves location information: shifted input $\rightarrow$ shifted feature map.

Partial translation invariance

For classification, we often care that a feature exists, not exactly where it is. Pooling, downsampling and later classification layers can build *some* local invariance.

Nuance

A standard CNN is not perfectly invariant to all translations.

Padding, stride and pooling



- **Padding** changes what happens at the border.
- **Stride** changes how far the convolution window jumps.
- **Pooling** changes a patch into a smaller summary map.

Dimension change

For input height H_{in} , kernel size is k , padding p , and stride s the output height is $H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1$ (and similarly for width).

Padding: add a zero border

What changes?

- Add zeros around the input before sliding the filter.
- Original edge pixels can be centres of full 3×3 windows.
- With $k = 3$, $p = 1$, $s = 1$, the spatial size is preserved.

Stride: jump in larger steps

What changes?

- Stride is the step size between neighbouring windows.
- Larger stride uses fewer windows.
- The output map is smaller: less computation, less spatial detail.

Pooling: summarise a local patch

What changes?

- Pooling has no learned weights.
- Max pooling keeps the strongest local activation.
- Average pooling uses the local mean instead.

A 2×2 pooling layer with stride 2 halves height and width.

The shape of a batch of images

A batch of N images is a four-dimensional array (a *tensor*), indexed by example, channel, height and width:

$$X \in \mathbb{R}^{N \times C \times H \times W}.$$

A **channel** is one 2D grid of values. Each feature map a layer produces becomes a channel for the next layer.

For the handwritten digits (one channel¹, 8×8 pixels) this is

$$X \in \mathbb{R}^{N \times 1 \times 8 \times 8}.$$

Grid, not vector

A fully connected network first *flattens* each image into a length-64 vector ($1 \cdot 8 \cdot 8$), discarding the grid. A convolution instead acts directly on the $1 \times 8 \times 8$ grid, so neighbouring pixels stay neighbours.

¹A grayscale image has one channel, an RGB image has three.

A convolutional layer: parameters and output size

Take a layer with C_{in} input channels, C_{out} filters (the output channels), kernel size k , padding p and stride s . Its weights form a tensor of shape

$$C_{\text{out}} \times C_{\text{in}} \times k \times k \quad (C_{\text{out}} \text{ filters, each } C_{\text{in}} \times k \times k),$$

so the number of trainable parameters is

$$C_{\text{out}} (C_{\text{in}} k^2 + 1),$$

one bias per output channel included.

As noted earlier, along each spatial dimension the output size is

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1.$$

A small CNN for digits

Building blocks of a CNN

A standard image classifier repeats a few simple operations:

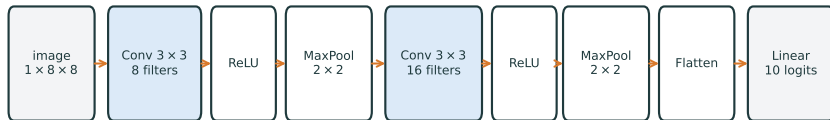


Interpretation

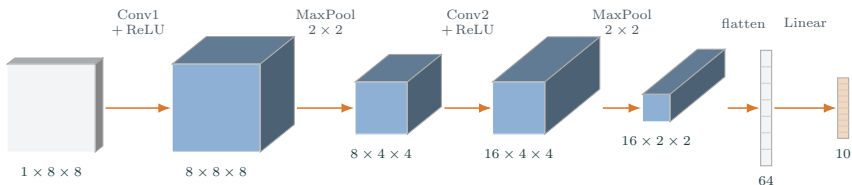
Early layers detect simple local patterns. Later layers combine these patterns into features useful for classification.

A small CNN architecture

A small convolutional neural network for digits



second pool gives $16 \times 2 \times 2$ features



Note: This is deliberately small for visualisation and quick training.²

²Throughout we set padding $p = 1$.

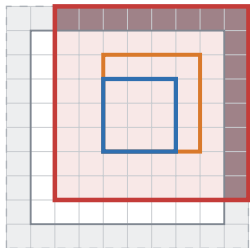
Shape walk-through

Layer	Operation	Output shape for one batch
Input	grayscale digit image	$(N, 1, 8, 8)$
Conv1	8 filters, 3×3 , padding 1	$(N, 8, 8, 8)$
ReLU	elementwise nonlinearity	$(N, 8, 8, 8)$
MaxPool	2×2 pooling	$(N, 8, 4, 4)$
Conv2	16 filters, 3×3 , padding 1	$(N, 16, 4, 4)$
ReLU	elementwise nonlinearity	$(N, 16, 4, 4)$
MaxPool	2×2 pooling	$(N, 16, 2, 2)$
Flatten	reshape features	$(N, 64)$
Linear	class scores	$(N, 10)$

Two things to notice

Conv2 has $C_{\text{in}} = 8$ because each filter sums over the eight feature maps from Conv1. The final output contains **10 row logits**, one for each digit class.

Deeper layers “see” more



input image (8×8)

conv1 3×3 pool1 4×4 conv2 7×7

For our network:



By conv2, one unit sees nearly the **whole digit**.

Receptive field

The *receptive field* of a unit is the region of the **original image** that can affect its value.

Filters stay 3×3 , yet the receptive field **grows with depth**: a 3×3 window of the previous map already summarises a larger image patch, and pooling spreads each step further.

Parameter count

For the small CNN:

$$\text{Conv1: } 8(1 \cdot 3 \cdot 3) + 8 = 80,$$

$$\text{Conv2: } 16(8 \cdot 3 \cdot 3) + 16 = 1168,$$

$$\text{Linear: } 64 \cdot 10 + 10 = 650.$$

Total:

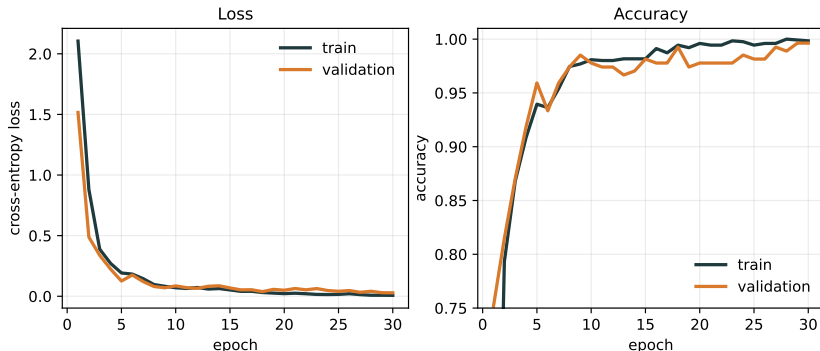
$$80 + 1168 + 650 = 1898 \text{ trainable parameters.}$$

Why this matters

CNNs often use far fewer parameters than comparable fully connected networks because filters are small and shared across locations. If it performs as well as a fully connected network, that's a huge parameter savings.

Training and validation curves

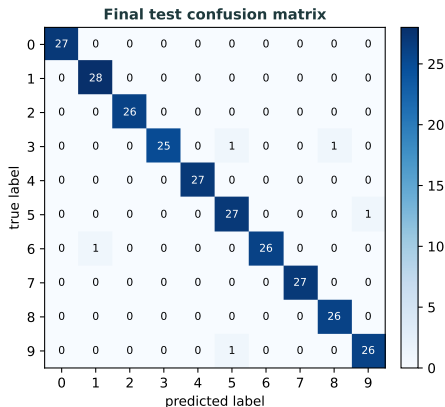
Training diagnostics for the small CNN



Recall

Training curves show optimisation progress. Validation curves help us monitor generalisation while developing the model.

Final test performance: confusion matrix



After model selection

Only after choices are fixed should we evaluate on the final test set.

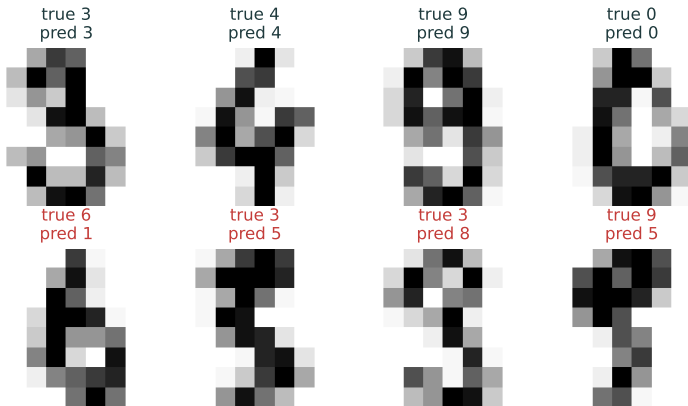
A confusion matrix shows which classes are mixed up.

Question

Which digit pairs have been confused? Does this match your visual intuition?

Inspecting predictions

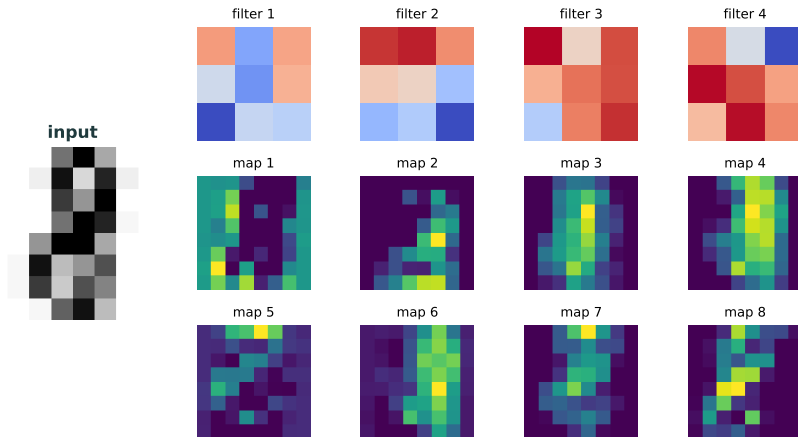
Inspect examples, especially mistakes



Looking at mistakes is often more informative than looking only at aggregate accuracy.

What did the first layer learn?

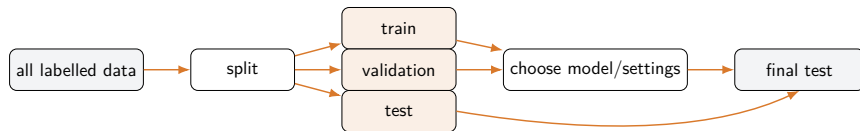
Learned filters and first-layer feature maps



The top row shows four learned 3×3 filters; the remaining panels show all eight first-layer activations on one image after training.

Practical CNN training

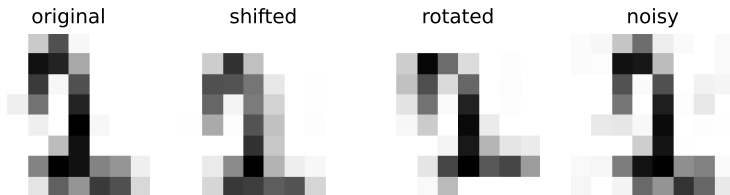
Same train/validation/test workflow



The test set should not be used repeatedly during model development.

Data augmentation

Data augmentation creates realistic transformed examples



Idea

Create realistic transformed training examples so the model learns invariances we expect to hold.

Examples include small shifts, small rotations, crops or brightness changes **if they preserve the label**. For digits, large rotations or flips can create unrealistic images or even change the apparent class.

Mini-exercise: think before coding

Suppose we replace the first convolutional layer in our example by one that has a 5×5 kernel and no padding.

Discuss:

- ① Does the output height/width increase, decrease, or stay the same?
- ② What padding would keep the spatial size at 8×8 ?
- ③ Does increasing the kernel size increase the number of parameters?

Answer sketch

Without padding, the output shrinks. Padding 2 keeps size for a 5×5 filter. The number of weights increases from $8 \cdot 1 \cdot 3 \cdot 3$ to $8 \cdot 1 \cdot 5 \cdot 5$, plus biases. Note that as the output size changes, the final linear layer may also need to change.

What comes next in the problem class?

The tutorial can now focus on implementation:

- load the digits dataset;
- train a small MLP from Lecture 9;
- train the small CNN from today;
- compare training curves and final test performance;
- inspect predictions and feature maps;
- optionally compare with PCA features from Lecture 8.

Goal

The aim is not to build a large model. The aim is to understand the training loop and the role of architecture.

Summary

- Images have spatial structure that fully connected layers ignore.
- CNNs use local filters shared across locations.
- A filter produces a feature map; multiple filters produce multiple channels.
- Padding, stride and pooling control spatial resolution.
- CNNs are trained with the same loss/backpropagation/optimiser loop as MLPs.
- When implementing CNNs, keep careful track of tensor shapes: (N, C, H, W) .
- Use validation data while developing and reserve the test set for final evaluation.